OpenMP – podstawy

Koszty współdzielenia danych w systemach równoległych Linux/Windows

Zadania-cele:

- uruchomienie prostych programów przy wykorzystaniu określonej wprost liczby wątków przetwarzania

- zastosowanie dyrektyw #pragma omp parallel, for, atomic

- zastosowanie funkcji omp: `omp_set_num_threads()`,`omp_get_thread_num()`

-pomiar czasu przetwarzania równoległego

- obserwacja kosztów współdzielenia danych

Zadanie 1 – prosty program w Open_MP (hello_world.c)

Kompilacja i uruchomienie Linux:

- gcc –fopenmp plikwe –o plikwyj -O3
- ./plikwyj
- Celem przełącznika –fopenmp jest uruchomienie kompilacji z uwzględnieniem dyrektyw OpenMP

Kroki pracy z kodem 1:

1. Wprowadzenie pliku nagłówkowego umożliwiającego pracę z funkcjami OpenMP:

- `#include <omp.h>`

2. Wprowadzenie do kodu dyrektywy zrównoleglającej przetwarzanie w **kolejnym po dyrektywie bloku kodu**:

- `#pragma omp parallel`

3. Zmienne prywatne i zmienne współdzielone wątków, obserwacja efektu wyścigu występującego w kodzie podczas modyfikacji zmiennych współdzielonych.

4. Ustalenie liczby uruchamianych wątków

- `omp_set_num_threads(liczba_uruchamianych_wątków)`

5. Uzyskanie informacji o wątku - źródle komunikatów pojawiających się na konsoli

- zastosowanie funkcji: `omp_get_thread_num()`

Obserwacja kolejności pojawiających się komunikatów świadczących o pracy równoległej przy użyciu wielu wątków przetwarzania (w zależności od określonej liczby wątków) – ewentualny niedeterminizm przetwarzania, .

Użycie dyrektywy podziału pracy w ramach pętli for:

- #pragma omp for
- Obserwacja numerów iteracji wykonywanych przez poszczególne wątki (określone identyfikatorem)
- Analiza efektów różnych wariantów dyrektywy podziału pracy **#pragma omp for** (porównaj wykład lub dokumentację OPENMP)
    - podział pracy domyślny
    - podział pracy statyczny blokowy
    - podział pracy statyczny cykliczny
    - podziłał pracy dynamiczny domyślny i sparametryzowany
    - podział dynamiczny - sterowany

Zadanie 2 - pomiar prędkości przetwarzania w funkcji użytej liczby wątków/procesorów dla różnych wersji kodu – obserwacja kosztów współdzielenia danych między wątkami. System komputera wielordzeniowego z pamięcią współdzieloną, środowisko Windows/Visual Studio lub Linux

- Dla wersji równoległych uruchomić przetwarzanie przy użyciu liczby wątków równej:
  - liczbie procesorów logicznych
  - liczbie procesorów fizycznych
- Dla każdej kolejnej wersji kodu równoległego PI2-PI6 należy:
  - porównać czas obliczeń z czasem wersji sekwencyjnej,
  - określić przyczyny wpływające na czasy przetwarzania, uwzględnić liczbę wątków i procesorów uczestniczących w przetwarzaniu oraz spodziewany **stopień lokalności dostępu** do danych/zmiennych,
  - określić czy występuje i dlaczego unieważnianie kopii linii pamięci podręcznej danych procesora przechowywanych w pamięciach podręcznych, czy jest to znaczące dla czasu przetwarzania?
  - dla systemu Linux porównać czas trwania obliczeń (wall clock) oraz czas użycia procesorów i wyjaśnić przyczyny uzyskanych wartości.
  - obliczyć przyspieszenie przetwarzania w wyniku zastosowania równoległości czyli iloraz czasu przetwarzania sekwencyjnego i czasu przetwarzania badanej wersji równoległej.

1. **Wersja pierwsza kodu PI1** - Wykorzystać **sekwencyjny** kod wyznaczenia wartości liczby pi metodą całkowania (Serial_pi.cpp). Całkowanie dotyczy funkcji okręgu o promieniu równym 1 umieszczonego w początku układu współrzędnych. Funkcja jest całkowana w zakresie argumentów od 0 do 1, całkowanie dotyczy obszaru ¼ koła o promieniu 1 położonego nad osią X. Obszar ten ma powierzchnię pi/4. Szukana liczba pi jest wyznaczona jako iloczyn wyznaczonego pola ¼ koła i liczby 4.

- Pomiaru czasu (Linux) **pracy procesorów** nad programem użytkownika (suma czasu pracy wszystkich użytych procesorów) za pomocą funkcji `clock_t` clock() (#include<time.h>), pomiar czasu w jednostkach będących częścią CLOCKS_PER_SEC sekundy
- Pomiaru **upływu czasu rzeczywistego** w ramach przetwarzania: funkcja omp_get_wtime() (#include <omp.h>) zwraca wartość typu double oznaczającą liczbę sekund, które upłynęły od arbitralnie wybranego momentu w przeszłości. Dwukrotne wywołanie funkcji pomiaru czasu i odjęcie otrzymanych wartości pozwala na określenie czasu, który upłynął między uruchomieniami funkcji (np. przed i po przetwarzaniu, którego czas jest mierzony).
- Liczba iteracji pętli – liczba kroków całkowania powinna umożliwić uzyskanie wyniku w czasie pojedynczych sekund. Kolejne eksperymenty (inne wersje kodu) wykonać dla tej samej liczby iteracji. W systemie Windows/Visual Studio należy wybrać generację kodu Release, w systemie Linux kompilator wprowadzić w tryb optymalizacji kodu przełącznikiem –O3

2. **Wersja druga kodu - PI2** Zrównoleglić obliczenia wyznaczania liczby PI wyłącznie poprzez dyrektywę utworzenia regionu równoległego #pragma omp paralel i dyrektywę podziału pracy `#pragma omp for` w ramach pętli for. Przeanalizować które zmienne są prywatne, a które współdzielone (czy współdzielenie dotyczy odczytu czy zapisu), zaobserwować wynik obliczeń (jakie mogą być przyczyny niepoprawnego wyniku?), jaki jest **stopień lokalności dostępu** do danych ? dlaczego ?, zapisać wariant kodu, czasy przetwarzania i wyliczyć przyspieszenie obliczeń.

3. **Wersja trzecia kodu - PI3** Zrealizować ciąg dostępów (odczyt, zapis) do współdzielonej sumy w sposób niepodzielny:

```
#pragma omp atomic
```

sum+=4.0/(1.+ x*x);

usunąć błędy współdzielenia pozostałych zmiennych, zaobserwować wynik obliczeń, zapisać wariant kodu, czasy przetwarzania i wyliczyć przyspieszenie obliczeń.. Proszę sprawdzić w opisie standardu Open MP (np. wykład OpenMP) jakie są konsekwencje umieszczenia w kodzie źródłowym dyrektywy **atomic** dla wynikowego kodu generowanego przez kompilator; jaki jest **stopień lokalności dostępu** do danych ? dlaczego ?

4. **Wersja czwarta kodu - PI4** Utworzyć i użyć zmienne prywatne każdego z wątków do zapamiętania prywatnego wyniku pracy oraz użyć dyrektywy #pragma omp atomic do zapewnienia poprawności scalenia w ramach zmiennej globalnej wyników pracy na prywatnej zmiennej - zaobserwować wynik obliczeń, jaki jest **stopień lokalności dostępu** do danych ? dlaczego ?, zapisać wariant kodu, czasy przetwarzania i wyliczyć przyspieszenie obliczeń.

5. **Wersja piąta kodu - PI5** Zrealizować automatycznie scalenie wartości obliczanych w prywatnych sumach częściowych - klauzula `reduction`, , jaki jest **stopień lokalności dostępu** do danych ? dlaczego ?, zapisać wariant kodu, czasy przetwarzania i wyliczyć przyspieszenie obliczeń.

6. **Wersja szósta kodu - PI6** Zaimplementować sumy częściowe wyznaczane przez poszczególne wątki w ramach współdzielonej przez wątki tablicy zmiennych typu `double` – każdy wątek modyfikuje jedno słowo w tablicy zapisując w nim swoją sumę częściową – modyfikowane przez różne wątki słowa są sąsiednimi elementami tablicy. Zaobserwować wzrost czasu przetwarzania spowodowany przez fakt fałszywego współdzielenia danych (ang. false sharing). Zjawisko to polega na wielokrotnym, cyklicznym (wywołanym przez wielokrotne zapisy w różnych procesorach wartości do tej samej linii pamięci podręcznej procesora):

- unieważnianiu powielonych linii pamięci w pp procesorów, które przestają być aktualne w wyniku zapisu danych do jednej z wielu kopii tej linii,
- konieczności sprowadzenia do pamięci podręcznej procesora realizującego kod wątku (korzystającego z danych sąsiednich) aktualnej kopii danych z unieważnionej linii.

Proszę zaobserwować wyniki i zapisać czas.
Przetwarzania **powinno** cechować się gorszą wydajnością niż wersja PI4. Wynika to ze spodziewanego dla tej wersji kodu unieważniania i konieczności sprowadzania do procesora aktualnej wersji linii danych pamięci podręcznej. Unieważnianie to jednak może nie wystąpić z jednego z dwóch powodów:

- przetwarzania wszystkich wątków na jednym fizycznym procesorze (2 procesory logiczne w technologii Hypertreading – jedna pamięć podręczna i jedna kopia linii pamięci dostępna przez wszystkie wątki),
- optymalizacji kodu przeprowadzanej przez kompilator. W wyniku tej optymalizacji może powstać kod wynikowy, w którym wartości zmiennych nie są zapisywane w pamięci, lecz pamiętane w rejestrach procesora do momentu zakończenia przetwarzania bloku kodu , w którym są wykorzystywane. Na wywołanie fałszywego współdzielenia można wpłynąć deklarując zmienną (której wartości mają być przechowywane w pamięci) przy użyciu dyrektywy volatile . volatile double tab[n] lub przez synchronizację obrazu prywatnego wątku dla zmiennej z obrazem tej zmiennej współdzielonym w pamięci - #pragma omp flush (tab); jaki jest **stopień lokalności dostępu** do danych ? dlaczego ?

7. **Wersja siódma kodu - PI7** Eksperyment z rozszerzoną wersją koduPI6 – celem eksperymentu jest wyznaczenie długości linii danych pamięci podręcznej procesora w oparciu o fałszywe współdzielenie.

Koncepcja eksperymentu polega na znalezieniu adresów w pamięci odpowiadających sąsiednim końcom linii pamięci podręcznej – wyznaczona odległość miedzy dwoma sąsiednimi końcami linii pp jest długością linii. Różnica adresów jest różnicą położenia wykorzystywanych słów zmiennej tablicowej zajmującej

ciągły obszar pamięci. Odległość między dwoma adresami słów zmiennej tablicowej wynika z rozmiaru tych słów (jeżeli korzystamy z elementu tab[5] i elementu tab[8] zmiennej tablicowej tab[n] typu double (8 bajtów) to niewykorzystany obszar pamięci (między tab[5] i tab[8] ma wielkość 16 bajtów).

W ramach tego eksperymentu wykorzystujemy:

- jedno uruchomienie kodu
- jedną zmienną tablicową zajmującą ok. 50 słów 8 bajtowych czyli kolejnych 400 bajtów pamięci
- dwa wątki pracujące (domyślnie) na dwóch procesorach fizycznych z różnymi pamięciami podręcznymi procesora (dla wywołania unieważnienia kopii powielonych linii pp).
- zakładamy, że słowo zmiennej będzie wyrównane do adresu początkowego linii pamięci (jeśli by tak nie było należy zrezygnować z wykorzystania sąsiednich słów, a użyć za każdym razem dwóch słów tablicy oddzielonych słowem niewykorzystanym).
- Realizujemy równolegle w kodzie obu wątków wiele odczytów i **zapisów** do sąsiednich słów tej samej tablicy przechowujących sumy częściowe algorytmu obliczania liczby pi.
- Przebieg eksperymentu:
  - Jedno uruchomienie kodu przy użyciu 2 wątków (wywołanie zapisów/odczytów przez volatile lub #pragma omp flush).
  - W ramach tego uruchomienia wątki wielokrotnie obliczają liczbę pi, przy użyciu za każdym razem innej pary słów z tablicy: tab[0] i tab[1]; tab[1]i tab[]2]; tab[2]i tab[3]; aż do przejścia przez całą tablicę.
  - Dla każdej iteracji przy użyciu kolejnej pary słów należy wyświetlić czas obliczeń wraz z numerem iteracji odpowiadającej użytym słowom – elementom tablicy
  - Pracując przy użyciu 2 wątków na dwóch słowach pamięci z jednej tablicy możemy uzyskać różne czasy przetwarzania zależnie od tego czy słowa użyte znajdują się w jednej czy dwóch liniach. Czas krótszy wystąpi wtedy gdy praca 2 wątków przebiega przy użyciu 2 różnych linii, nie ma wtedy unieważnienia kopii linii pamięci podręcznej procesora. Czas krótszy określa zatem położenie jednego z końców linii. Znalezienie sąsiedniego miejsca w pamięci (poprzez wykorzystanie kolejno sąsiednich słów tablicy), któremu odpowiada krótszy czas obliczeń pozwala na znalezienie położenia kolejnego końca linii pamięci podręcznej. Odległość między kolejnymi końcami odpowiada poszukiwanej długości linii pamięci podręcznej procesora. Krótsze przetwarzanie będzie się powtarzać cyklicznie tak jak cyklicznie występują końce linii. Długość linii będzie podana wzorem zależnym od występujących w cyklu liczby iteracji o dłuższym czasie przetwarzania, liczby iteracji o krótszym czasie przetwarzania oraz wielkości słowa w pamięci.

- Możliwe przyczyny braku powodzenia eksperymentu to niespełnione warunki konieczne:
  - ciągły obszar danych – jedna tablica przez wszystkie iteracji obliczeń,
  - dwa wątki przetwarzania,
  - dwa procesory fizyczne – dwie pamięci podręczne procesora,
  - zapisy do pamięci wywołujące fałszywe współdzielenie (volatile/flush),
  - brak wyrównania słowa użytej tablicy do brzegu linii.

Opracowanie wyników eksperymentu to:

1. Tabela zawierająca dla kodów PI1 - PI6:
   - wyniki upływającego czasu rzeczywistego podczas obliczeń,
   - współczynnik przyspieszenia obliczeń dla uruchomień równoległych kodu i określonej liczby wątków (iloraz: czas sekwencyjny/czas równoległy)
   - uzasadnienie dla uzyskanej wielkości przyspieszenia – dlaczego czas przetwarzania równoległego jest dłuższy/ krótszy od czasu przetwarzania sekwencyjnego – proszę w wyjaśnieniach skorzystać z wykładu o zagadnieniach dotyczących współdzielenia pamięci w systemach równoległych i OpenMP.

2. Określenie długości linii wg przygotowanego wzoru i na podstawie wyników eksperymentu z kodem PI7 oraz uzasadnienie.
3. Dostępność wersji kodu i umiejętność uzasadnienia uzyskanych dla uruchomień tych kodów wyników braku poprawności przetwarzania, czasu przetwarzania procesorów, upływu czasu rzeczywistego podczas obliczeń.

Materiały pomocnicze to wykłady: pamięć w systemach wieloprocesorowych z pamięcią współdzieloną, wykład/dokumentacja OpenMP.

Uwaga zakres wiadomości zadania (materiał wykładowy: pamięć współdzielona i OpenMP) jest zaliczany sprawdzianem na drugich zajęciach.

Przygotowano: 8.10.2025